

Know VAT & ZATCA — RAG Ingestion Pipeline

Debugging session, root causes, fixes, and working commands — 2026-07-23

Contents

- 1. Context
- 2. Problems found (6)
- 3. Code fixes
- 4. Infrastructure note (Qdrant network)
- 5. Working commands reference
- 6. Final verified state
- 7. Known limitations / follow-ups

1. Context

Feature A ("Know VAT & ZATCA") is a RAG knowledge base over 16 ZATCA/SOCPA source documents (VAT law, GCC agreement, e-invoicing standards, zakat regulations, etc.). An earlier session ran the offline ingestion pipeline (2026-07-22) and got 412 Qdrant points — it looked like a mostly-successful run, with one document flagged in `retry_manifest.json` for a transient failure.

This session started from one specific complaint — `vat_implementing_regulations_full_text.pdf` wasn't chunking — and turned into a full audit of all 16 documents, because the same class of bug (silent data loss with zero errors logged) turned out to affect most of the corpus, not just the one document. The guiding instruction throughout: **don't lose any data**.

2. Problems found

1 Word-form article numbers

`vat_implementing_regulations_full_text.pdf` spells some article numbers as words instead of digits — real heading found: `ARTICLE FORTY-THREE: COLLECTION OF TAX ON IMPORTS`. Body citations do the same: *"pursuant to Article forty-six of these Regulations"*. The old regex `(Article\s+\(?([0-9]+)\)?\s*: )` only matched digits, so it silently matched nothing in this document — confirmed by checking Qdrant directly: this document had **zero** prose chunks after the original run, only 5 leftover table chunks.

2 One flat-text regex can't cover every document's heading style

Real documents format article headings completely differently:

Document	Real heading format	Old regex result
VAT Implementing Regulations	Bare ALL-CAPS titles, no "Article N" text at all for ~90% of articles	0 matches
GCC Unified VAT Agreement	<code>Article (35)</code> alone on its own line, title on next line, no colon	0 matches

Fix: switched from flat-text regex matching to using **Docling's markdown headings** (lines starting with `#`) as the primary boundary signal — Docling's layout model already separates real section titles from body prose, which a regex on flat text structurally cannot do reliably.

3 Docling doesn't promote every article marker to a real heading

Found while validating the heading-based fix against `socpa_accountants_regulations.pdf`: Docling only promoted **2 of 38** "Article (N) :" markers to actual markdown headings — the rest sit as plain inline text inside long flowing paragraphs. Heading-only detection merged all 36 un-promoted articles into a single oversized "Article 2" chunk (confirmed: 8 chunks in Qdrant, all labeled `article_number: "2"`).

FIX

Hybrid boundary detection: markdown headings + a flat-text scan for colon-terminated `Article N:` mentions found *outside* any heading span. The colon is the signal that distinguishes a real marker ("Article (24) :") from a mid-sentence citation ("...subject to Article 25.", no colon). Re-validated against VAT Implementing Regulations and GCC after this change — both unchanged (98 and 66 chunks respectively), SOCPA went from 8 to 38 chunks (exactly 1 per real article).

4 GCC Unified VAT Agreement mislabeled language

`gcc_unified_vat_agreement_ar.pdf` was gathered from the `/ar/` URL path and manifested as `language: "ar"`, but its actual extracted content is **English** (headings read "Article (2)", "Article (3) Calculation of Dates", etc. — only 3 stray Arabic characters in the whole 1183-line extraction). The Arabic-language regex (`المادة`) was being run against English text, matching nothing.

FIX

Corrected `language` to `"en"` in `manifest.json`. Flagged (not silently reconciled) in `SOURCE_MANIFEST.md` — a real Arabic version of this agreement is still not gathered if one is actually needed later.

5 Non-article documents lost 100% of their content, silently

Any document with no "Article N" structure at all — hub overview pages (`VAT Hub Overview`, `E-Invoicing Hub Overview`, `Zakat Regulations Hub`) and guideline-style documents organized by decimal section numbers instead of articles (`VAT Implementing Regulations Amendments`, and it turned out several documents originally assumed to be "fine" too: `XML Implementation Standard`, `Security Features Implementation Standard`, `Detailed E-Invoicing Guideline`, `Guideline for Zakat Documents`, `Zakat Collection Overview` — these are technical standards, not article-numbered legal text) — produced zero prose chunks with **zero errors logged**. `ingest.py` treated an empty chunk list as a normal, successful outcome.

EXAMPLE — REAL CONTENT THAT WAS SILENTLY DISCARDED

`vat_hub_overview.pdf`, full extracted text (this entire page, verbatim, produced **zero** Qdrant chunks under the old code):

VAT Hub Overview and SME Compliance Simplifications

Value Added Tax (VAT) is an indirect tax imposed on all goods and services that are bought and sold by businesses, with a few exceptions. VAT is applied in more than 160 countries around the world as a reliable source of revenue for state budgets.

Facilitating SMEs compliance with VAT:

Tax Return Filing: Filing an annual tax return for establishments whose annual taxable supplies do not exceed 40 million SAR.

Simplified Invoicing: Using a simplified tax invoice that includes the issuance date, supplier name and address, item/service description, payment amount, and VAT value...

Cash Basis Accounting: Using the cash basis accounting method for establishments whose annual supplies (current or previous year) do not exceed 5 million SAR.

...

No exception, no log line calling this out — `ingest.py` logged `Ingested vat_hub_overview.pdf: {'prose_chunks': 0, 'tables': 0}` as if that were a completed, successful ingestion.

FIX

Added `chunk_fallback()` in `chunker.py`: chunks by Docling's own markdown headings when the document has any, otherwise groups blank-line-separated paragraphs up to the word cap. Wired into `ingest.py`: if the normal article chunker returns nothing for a non-empty document, fall back instead of silently dropping the content. After the fix, this exact page produced 1 real chunk; documents with real heading structure (the guideline- style ones) produced far more — e.g. Zakat Collection Overview went from 0 to 137 prose chunks, Detailed E-Invoicing Guideline from 0 to 129. Hundreds of previously invisible chunks recovered across the corpus, not just the originally flagged documents.

6 Long articles silently overwrote themselves in Qdrant

Unrelated to everything above, and pre-existing in the code before this session. Any article whose text exceeded `MAX_ARTICLE_WORDS` (500) got split into multiple pieces (`chunk_document` 's sub-split logic), but the Qdrant point ID for every piece was generated from just (`document_type`, `version_label`, "article", `article_number`) — **identical for every piece**. Each piece's `upsert` silently overwrote the previous one, so only the *last* piece of any long article ever survived in Qdrant.

CONFIRMED

Article 79 of VAT Implementing Regulations ("Transitional Provisions") split into 3 parts during chunking; only (part 3/3) was present in Qdrant — the first ~1,000 words of the article were gone from the vector store (source PDF itself was never touched, just the derived index).

FIX

Added a `part_index` field to every chunk dict (both `chunk_document` and `chunk_fallback`), and included it in the point ID generation in `ingest.py`:

```
"id": _stable_id(
    document_type, version_label, "article", chunk["article_number"],
    str(chunk.get("part_index", 0)),
),
```

Because this changed the ID scheme for every prose chunk (not just split ones), a full wipe-and-rebuild was needed rather than a targeted re-ingest — old IDs would have become orphaned duplicates otherwise.

Also found: duplicate table rows from retry attempts

`sqlite_store.insert_table_rows()` had no dedup — a plain `INSERT`, no delete-first. My own retries during this session (a run that failed partway through — got as far as writing table rows to SQLite before hitting the Qdrant network issue in §4 — followed by a successful retry that ingested the same document again) left 3x duplicate rows in 8 tables before this was caught and cleaned up.

EXAMPLE — REAL ROW COUNTS BEFORE CLEANUP

table_ref	Rows before dedup	Correct row count
vat_implementing_regulations_table_0	9	3
vat_implementing_regulations_table_1	12	4
vat_implementing_regulations_table_3	6	2
vat_implementing_regulations_amendments_table_0	66	22
vat_implementing_regulations_amendments_table_1	15	5

Every affected table had exactly **3x** its real row count — one copy from the original 2026-07-22 run, plus one copy from each of this session's two ingestion attempts for that document. Diagnosed with:

```
SELECT table_ref, row_index, COUNT(*) c FROM table_rows
GROUP BY table_ref, row_index HAVING c > 1;
```

FIX

`insert_table_rows()` now does `DELETE FROM table_rows WHERE table_ref = ?` before inserting — re-ingesting a document is now a safe, idempotent operation. Same class of bug as the one already fixed in `work.customer_memories` during the auth work (missing upsert).

3. Code fixes — files touched

File	Change
<code>agents/known_vat_n_zatca/offline/chunker.py</code>	English word-number normalizer; heading-based + hybrid flat-colon boundary detection (<code>split_into_articles_en</code>); ALL-CAPS heuristic to reject spurious Docling headings; new <code>chunk_fallback()</code> for non-article documents; <code>part_index</code> added to every chunk dict.
<code>agents/known_vat_n_zatca/offline/references.py</code>	Citation regex extended to catch word-form and parenthesized article numbers (<code>Articles?s*\(?([A-Za-z-]+ [0-9]+)\)?</code>), reusing the chunker's normalizer.
<code>agents/known_vat_n_zatca/offline/ingest.py</code>	Falls back to <code>chunk_fallback()</code> when the article chunker returns nothing for a non-empty document; point ID generation now includes <code>part_index</code> .
<code>agents/known_vat_n_zatca/sqlite_store.py</code>	<code>insert_table_rows()</code> deletes existing rows for a <code>table_ref</code> before inserting (idempotent re-ingestion).
<code>docs/rag_ingestion_data/manifest.json</code>	GCC Unified VAT Agreement language corrected <code>ar</code> → <code>en</code> ; merged in the former <code>retry_manifest.json</code> entry (Zakat Regulations Hub) — now the single 16-document source of truth. <code>retry_manifest.json</code> deleted (resolved).
<code>docs/rag_ingestion_data/SOURCE_MANIFEST.md</code>	Added a note documenting the GCC language mismatch finding — flagged, not silently reconciled.

4. Infrastructure note: Qdrant network attachment

NOT A CODE BUG — ENVIRONMENT FINDING

mizan-qdrant is **not actually on the mizanai_default compose network** — only mizan-postgres is. The offline ingestion Dockerfile's documented example (`--network mizan-ai_default` , note the hyphen — also slightly wrong, the real network is `mizanai_default`) doesn't reach Qdrant at all. Confirmed via:

```
docker network inspect mizanai_default --format '{{range .Containers}}{{.Name}}
{{.IPv4Address}}{{"\n"}}{{end}}'
# -> only mizan-postgres

docker inspect mizan-qdrant --format '{{range $k,$v := .NetworkSettings.Networks}}{{$k}}
{{$v.IPAddress}}{{"\n"}}{{end}}'
# -> bridge 172.17.0.2 (default bridge, not the compose network)
```

Worked around this session using `QDRANT_URL=http://host.docker.internal:6333` instead (Qdrant's port 6333 is published to the host anyway, so any container can reach it regardless of network membership). Worth deciding later whether to fix the compose network attachment properly or just standardize on `host.docker.internal` going forward.

5. Working commands reference

5.1 Rebuild the offline ingestion image (after any code change)

```
docker build -f services/know_vat_n_zatca_offline/Dockerfile -t know-vat-zatca-offline .
```

GOTCHA

Docker's build cache reused a 10-hour-stale `COPY . .` layer at least once this session despite source files having genuinely changed. If a rebuilt image doesn't seem to reflect recent edits, don't trust it blindly — either force a clean build or bind-mount the source over the image at run time (see 5.2), which sidesteps the caching question entirely.

5.2 Run ingestion (the reliable pattern used to close out this session)

Bind-mounts `agents/` and `manifest.json` directly from the host, so the container always runs the current on-disk code regardless of what got baked into the image at build time. Run from the repo root, in Git Bash on Windows (`MSYS_NO_PATHCONV=1` stops Git Bash mangling the container-side `/app/...` paths):

```
MSYS_NO_PATHCONV=1 docker run --rm --gpus all \
  -e QDRANT_URL=http://host.docker.internal:6333 \
  -v mizanai_know_vat_zatca_sqlite:/data \
  -v "C:\Users\kalim\models\bge-m3:/models/bge-m3" \
  -v "<repo>\agents:/app/agents" \
  -v "
<repo>\docs\rag_ingestion_data\manifest.json:/app/docs/rag_ingestion_data/manifest.json" \
  know-vat-zatca-offline \
  python -m agents.know_vat_n_zatca.offline.ingest \
    --manifest docs/rag_ingestion_data/manifest.json \
    --embedding-model-path /models/bge-m3
```

To re-ingest only a subset of documents, point `--manifest` at a scoped JSON file containing just those entries (same shape as `manifest.json`, a list of `{path, document_type, language, version_label, is_current, source_site, source_url}`) and bind-mount that file instead. Only do this for documents whose chunking behavior didn't change elsewhere in the corpus — a boundary-detection change (like problem #3 above) can affect documents you didn't touch, so when in doubt, re-ingest everything.

5.3 Wipe Qdrant collection + SQLite tables (clean rebuild)

Destructive — only do this when you're confident the pipeline is correct and want a guaranteed clean state (e.g. after an ID-scheme change like problem #6, where stale points would otherwise become orphans). Get explicit sign-off before running this against anything that matters.

```
# PowerShell (Bash may block DELETE-shaped commands via the auto-mode classifier)
Invoke-RestMethod -Uri "http://localhost:6333/collections/know_vat_n_zatca" -Method Delete

docker run --rm -v mizanai_know_vat_zatca_sqlite:/data python:3.10-slim python -c "
import sqlite3
con = sqlite3.connect('/data/know_vat_n_zatca.db')
cur = con.cursor()
cur.execute('DELETE FROM table_rows')
cur.execute('DELETE FROM tables_registry')
con.commit()
"
```

The collection recreates itself automatically on the next ingestion run (`qdrant_store.ensure_collection()`), with the same config: 1024-dim vectors, cosine distance.

5.4 Verify a connection / debug the pipeline without a full run

```
# Confirm host.docker.internal reaches Qdrant from inside a container
docker run --rm curlimages/curl:latest curl -s -m 5
http://host.docker.internal:6333/collections

# Inspect the real SQLite volume directly (no ingestion needed)
docker run --rm -v mizanai_know_vat_zatca_sqlite:/data python:3.10-slim python -c "
import sqlite3
con = sqlite3.connect('/data/know_vat_n_zatca.db')
cur = con.cursor()
cur.execute('SELECT COUNT(*) FROM tables_registry'); print(cur.fetchone())
cur.execute('SELECT COUNT(*) FROM table_rows'); print(cur.fetchone())
"

# List and inspect docker volumes / networks (useful when something can't connect)
docker volume ls
docker volume inspect mizanai_know_vat_zatca_sqlite
docker network ls
docker network inspect mizanai_default
docker inspect mizan-qdrant --format '{{json .NetworkSettings.Networks}}'
```

5.5 Query Qdrant directly (no Python needed)

```
# Point count for the collection
curl -s http://localhost:6333/collections/know_vat_n_zatca

# Points for one document, filtered by payload field
curl -s -X POST http://localhost:6333/collections/know_vat_n_zatca/points/scroll \
  -H "Content-Type: application/json" \
  -d '{"filter": {"must": [{"key": "document_type", "match": {"value": "SOCPA Accountants Regulations"}}]}, "limit": 50, "with_payload": true}'
```

GOTCHA

Qdrant's `/points/scroll` caps each response (this collection defaults to 500-ish per page) — always check `next_page_offset` in the response and keep paginating with it as the `offset` field on the next request. A single un-paginated scroll under-reported the true point count by ~25% during this session's final verification.

5.6 Local (non-container) testing during development

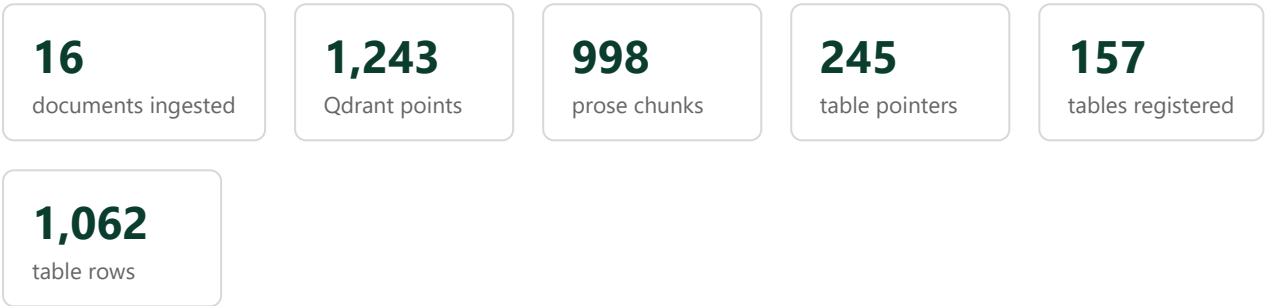
Useful for quickly validating a `chunker.py` change against already-extracted Docling markdown without re-running the whole slow OCR pipeline. The heavy offline deps (torch, docling, FlagEmbedding) are installed in a dedicated conda env, not the base Python:


```
# Confirmed working env for local Docling/embedding testing (2026-07-21)
C:\Users\kalim\miniconda3\envs\torchEnv
# torch 2.13.0+cu126, RTX 4060 Laptop GPU

"/c/Users/kalim/miniconda3/envs/torchEnv/python.exe" -c "import torch;
print(torch.__version__, torch.cuda.is_available())"

# BGE-M3 checkpoint, referenced by --embedding-model-path in the container:
C:\Users\kalim\models\bge-m3
```

6. Final verified state (2026-07-23)



Every one of the 15 unique document types (16 manifest entries — XML Implementation Standard has two versions) has at least one Qdrant point. Zero documents with zero chunks. Spot-checked article counts against known ground truth: SOCPA 38/38 real articles, VAT Implementing Regulations 79/79.

7. Known limitations / follow-ups (not chased further this session)

- **GCC Article 5** never got a detectable heading in Docling's output for that specific document (its neighbors, Article 4 and Article 6, are both real headings) — its content is present but merged into Article 4's chunk rather than being its own citable article. Minor, acceptable gap.
- **Table row counts vary slightly between identical re-runs** (1437 rows in one run, 1062 in another, for the same 157 tables) — Docling's OCR-based table structure recognition isn't fully deterministic. Not a code bug; just a characteristic of OCR-dependent extraction worth knowing about if row counts ever look "off" after a future re-ingest.
- **Qdrant network attachment** (§4) — works via `host.docker.internal` , but the underlying compose network gap hasn't been fixed at the infrastructure level.
- The two Claude-API-driven offline steps (real table descriptions, cross-reference Claude review pass) remain intentionally spec-only per `agents/know_vat_n_zatca/offline/CLAUDE_STEPS_SPEC.md` — unrelated to this session's fixes, not touched.